Atelier Machine Learning

Analyse Comportementale de la Clientèle Retail

COMPTE RENDU FINAL

Atelier Pratique : Prédiction du churn client

Préparé par :

Yassine Meftah

Année Universitaire : 2025-2026

Résumé

Ce projet s'inscrit dans le cadre de l'analyse comportementale de la clientèle d'une plateforme de e-commerce. L'objectif principal est de construire une chaîne complète de Machine Learning capable de prédire le churn client, d'explorer la structure des données, de segmenter les profils clients et de proposer une solution d'inférence exploitable. Un pipeline complet a été mis en place : préparation des données, parsing des dates, feature engineering, détection et correction des valeurs aberrantes, suppression des colonnes redondantes ou fuyantes, rééquilibrage des classes par SMOTENC, réduction de dimension par ACP, clustering KMeans, classification supervisée, régression et validation croisée. Le meilleur modèle de classification obtenu est un `RandomForest_GridSearch`. Le meilleur modèle de régression obtenu est un `HistGradientBoostingRegressor`. Une phase d'inférence a également été réalisée via un script de prédiction et une interface Flask. Le projet met ainsi en œuvre une démarche complète, reproductible et proche d'un cas réel de mise en production.

Abstract

This project focuses on customer behavioral analysis in a retail e-commerce setting. The main objective is to build a complete Machine Learning pipeline capable of predicting customer churn, exploring the structure of the data, segmenting customer profiles, and providing a reusable inference solution. A full workflow was implemented : data preparation, date parsing, feature engineering, outlier handling, removal of redundant and leakage-prone variables, class rebalancing with SMOTENC, dimensionality reduction with PCA, KMeans clustering, supervised classification, regression, and cross-validation. The best classification model is a `RandomForest_GridSearch`, while the best regression model is a `HistGradientBoostingRegressor`. An inference phase was also developed through a prediction script and a Flask interface. Overall, this project demonstrates a complete, reproducible workflow close to a real deployment scenario.

Table des matières

1 Introduction générale **4**
1.1 Contexte 4
1.2 Problématique 4
1.3 Objectifs du projet 4
1.4 Démarche suivie 4

2 Présentation du jeu de données **4**
2.1 Description générale 4
2.2 Typologie des variables 4
2.3 Problèmes de qualité identifiés 5

3 Analyse exploratoire et compréhension des données **5**
3.1 Qualité des données 5
3.2 Corrélations et multicolinéarité 5
3.3 Lecture métier 6

4 Préparation des données **6**
4.1 Nettoyage initial . . . 6
4.2 Détection de fuite d'information . . . 7
4.3 Parsing et transformation des dates . . . 7
4.4 Feature engineering . . . 7
4.5 Suppression de variables redondantes . . . 7
4.6 Gestion des outliers . . . 7
4.7 Gestion des valeurs manquantes . . . 7
4.8 Encodage et normalisation . . . 8
4.9 Rééquilibrage des classes . . . 8
4.10 Jeu final utilisé . . . 8

5 Réduction de dimension et segmentation **8**
5.1 Analyse en composantes principales (PCA) . . . 8
5.2 Projection 2D . . . 9
5.3 Segmentation KMeans . . . 9
5.4 Interprétation . . . 10

6 Modélisation prédictive : classification **10**
6.1 Objectif . . . 10
6.2 Modèles testés . . . 11
6.3 Métriques d'évaluation . . . 11
6.4 Résultats de classification . . . 11
6.5 Matrice de confusion . . . 12
6.6 Interprétation . . . 12
6.7 Importance des variables . . . 13

7 Modélisation prédictive : régression **13**
7.1 Objectif . . . 13
7.2 Modèles testés . . . 13
7.3 Métriques . . . 13
7.4 Résultats . . . 14
7.5 Analyse critique . . . 14

8 Validation croisée et robustesse **14**
8.1 Validation croisée en classification . . . 14
8.2 Validation croisée en régression . . . 14
8.3 Interprétation . . . 15

9 Prédiction et inférence **15**
9.1 Rôle du script de prédiction . . . 15
9.2 Pipeline d'inférence . . . 15
9.3 Résultats observés . . . 15

10 Déploiement avec Flask **15**
10.1 Objectif . . . 15
10.2 Architecture . . . 15
10.3 Modes d'utilisation . . . 16

10.4 Problèmes rencontrés 16
10.5 Corrections apportées 16
10.6 Limites 16

11 Discussion générale **16**
11.1 Points forts du projet 16
11.2 Limites 17
11.3 Améliorations possibles 17

12 Conclusion générale **17**

1 Introduction générale

1.1 Contexte

Dans le domaine du retail et du e-commerce, l'analyse des données clients constitue un levier stratégique pour améliorer la fidélisation, personnaliser les campagnes marketing et optimiser la rentabilité. La capacité à identifier les clients susceptibles de quitter la plateforme permet d'anticiper les actions commerciales et de réduire la perte de valeur.

1.2 Problématique

Le churn client représente un enjeu majeur pour les entreprises, car il est généralement plus coûteux d'acquérir un nouveau client que de conserver un client existant. Le problème traité dans ce projet consiste donc à exploiter les données comportementales et transactionnelles afin de prédire si un client présente un risque de départ.

1.3 Objectifs du projet

Le projet poursuit plusieurs objectifs complémentaires :

- développer un modèle de classification pour prédire le churn ;
- analyser la structure des données et identifier les variables importantes ;
- appliquer une réduction de dimension par ACP pour la visualisation et le clustering ;
- segmenter les clients via KMeans ;
- entraîner un modèle de régression pour prédire `MonetaryTotal` ;
- déployer une solution d'inférence réutilisable via un script Python et une interface Flask.

1.4 Démarche suivie

La méthodologie suivie s'organise selon une chaîne complète :

Préparation des données → Transformation → Modélisation → Évaluation → Déploiement

2 Présentation du jeu de données

2.1 Description générale

Le dataset étudié correspond à une base client retail contenant des variables numériques, catégorielles et dérivées. Il inclut des informations transactionnelles, temporelles, démographiques et comportementales. La variable cible principale du projet est `Churn`.

2.2 Typologie des variables

Les variables peuvent être regroupées en plusieurs familles :

- **Variables transactionnelles :** `Frequency`, `MonetaryTotal`, `MonetaryAvg`, `MonetaryStd`, `UniqueProducts`, `ReturnRatio`, etc.
- **Variables temporelles :** `PreferredHour`, `PreferredMonth`, `RegYear`, `RegMonth`, `RegDay`, etc.
- **Variables de relation client :** `SupportTicketsCount`, `SatisfactionScore`.
- **Variables démographiques et contextuelles :** `Age`, `Gender`, `Country`, `Region`.

2.3 Problèmes de qualité identifiés

L'analyse initiale du jeu de données a mis en évidence :

- des valeurs manquantes, notamment sur `Age` ;
- un format de date à normaliser pour `RegistrationDate` ;
- la présence de valeurs aberrantes sur certaines colonnes ;
- des variables redondantes ;
- des variables potentiellement fuyantes par rapport à la cible ;
- un déséquilibre entre les classes `Churn = 0` et `Churn = 1`.

3 Analyse exploratoire et compréhension des données

3.1 Qualité des données

La première étape a consisté à examiner la qualité globale du jeu de données, notamment la présence de valeurs manquantes, de colonnes peu informatives, de formats incohérents et de variables susceptibles de dégrader l'apprentissage.

3.2 Corrélations et multicolinéarité

Une analyse des corrélations entre variables numériques a été réalisée afin d'identifier les dépendances fortes et les redondances.

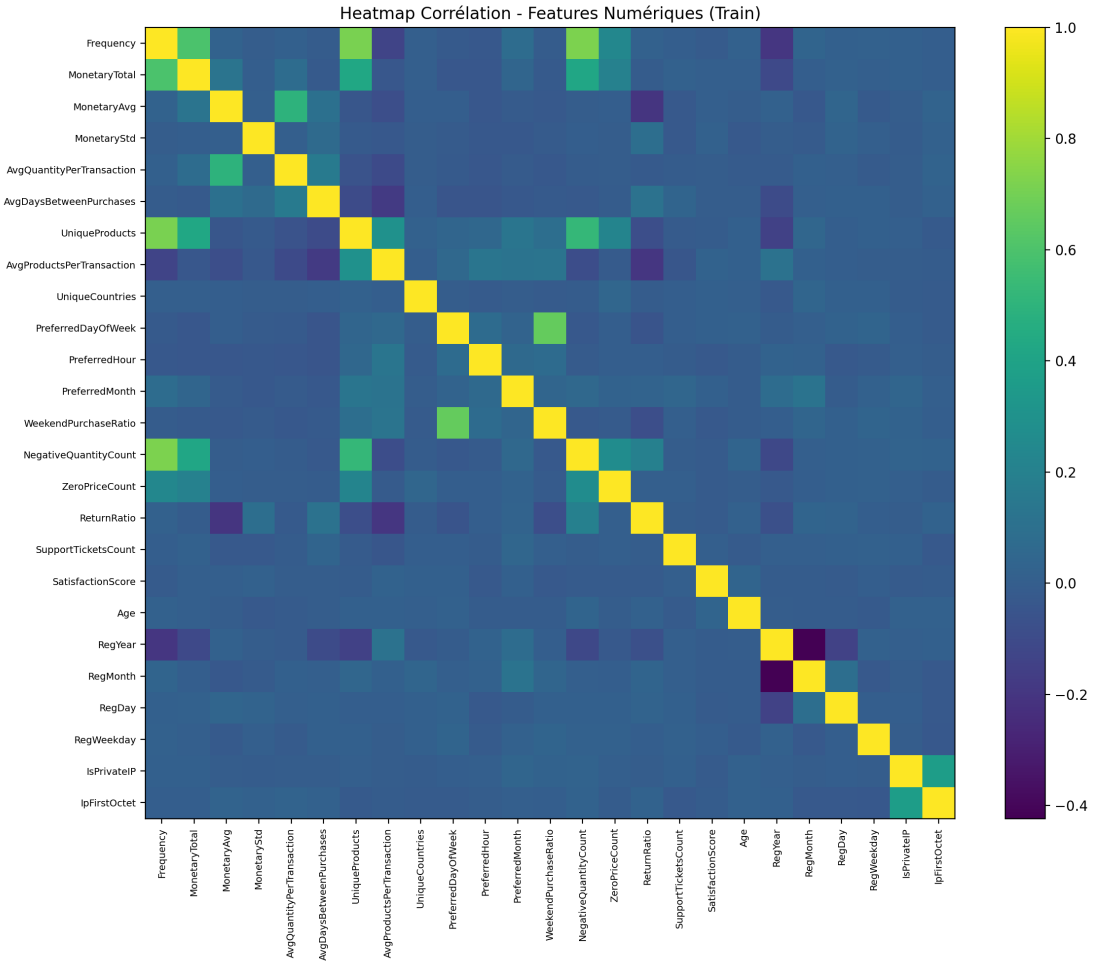

FIGURE 1 – Matrice de corrélation des variables numériques après nettoyage.

Cette heatmap montre une structure plus saine après nettoyage : certaines corrélations modérées demeurent, ce qui est normal, mais les redondances les plus problématiques ont été réduites.

3.3 Lecture métier

Plusieurs variables ont une signification métier directe :

- `Frequency` mesure l'intensité d'activité d'un client ;
- `MonetaryTotal` traduit la valeur économique générée ;
- `UniqueProducts` et `AvgProductsPerTransaction` décrivent la diversité et la richesse du panier ;
- `SupportTicketsCount` et `SatisfactionScore` renseignent sur la qualité de la relation client.

4 Préparation des données

4.1 Nettoyage initial

Plusieurs colonnes ont été supprimées dès le départ :

- les identifiants non prédictifs ;
- les variables constantes ;
- les variables fuyantes ou trop proches de la cible ;
- certaines variables redondantes détectées lors de l'analyse exploratoire.

4.2 Détection de fuite d'information

Une étape spécifique d'audit a été menée pour identifier les colonnes suspectes vis-à-vis de la cible, telles que :

- `AccountStatus`
- `LoyaltyLevel`
- `SpendingCategory`
- `BasketSizeCategory`
- `AvgBasketValue`

L'objectif était de vérifier si certaines colonnes reconstruisaient implicitement la cible ou en contenaient une approximation trop forte.

4.3 Parsing et transformation des dates

La variable `RegistrationDate` a été convertie puis décomposée en plusieurs composantes :

- `RegYear`
- `RegMonth`
- `RegDay`
- `RegWeekday`

4.4 Feature engineering

Un travail de feature engineering a été réalisé sur :

- les adresses IP : création de `IsPrivateIP` et `IpFirstOctet` ;
- les variables comportementales existantes ;
- la structuration finale d'un ensemble de variables robuste et interprétable.

4.5 Suppression de variables redondantes

Des variables redondantes ont été supprimées afin de simplifier l'espace de représentation et limiter la duplication d'information. Cette étape repose à la fois sur l'analyse statistique et sur une lecture métier.

4.6 Gestion des outliers

Les valeurs aberrantes ont été traitées via la méthode IQR. Les bornes ont été calculées sur le jeu d'entraînement uniquement, puis réappliquées au jeu de test afin d'éviter toute fuite d'information.

4.7 Gestion des valeurs manquantes

Le pipeline de prétraitement combine plusieurs stratégies :

- médiane pour la plupart des variables numériques ;
- mode pour les variables catégorielles ;
- `KNNImputer` spécifiquement pour `Age`.

4.8 Encodage et normalisation

Le prétraitement final repose sur un `ColumnTransformer` combinant :

- `SimpleImputer` + `StandardScaler` pour les variables numériques ;
- `KNNImputer` + `StandardScaler` pour `Age` ;
- `SimpleImputer` + `OneHotEncoder` pour les variables catégorielles.

4.9 Rééquilibrage des classes

Le jeu d'entraînement a été rééquilibré par `SMOTENC`. Cette opération n'a été appliquée que sur le train, afin de respecter une méthodologie correcte.

4.10 Jeu final utilisé

Le jeu final utilisé pour la classification comporte 33 variables conservées après nettoyage, sélection et suppression des colonnes à risque de fuite.

5 Réduction de dimension et segmentation

5.1 Analyse en composantes principales (PCA)

Une ACP automatique a été appliquée après prétraitement afin de réduire la dimensionnalité tout en conservant l'essentiel de l'information.

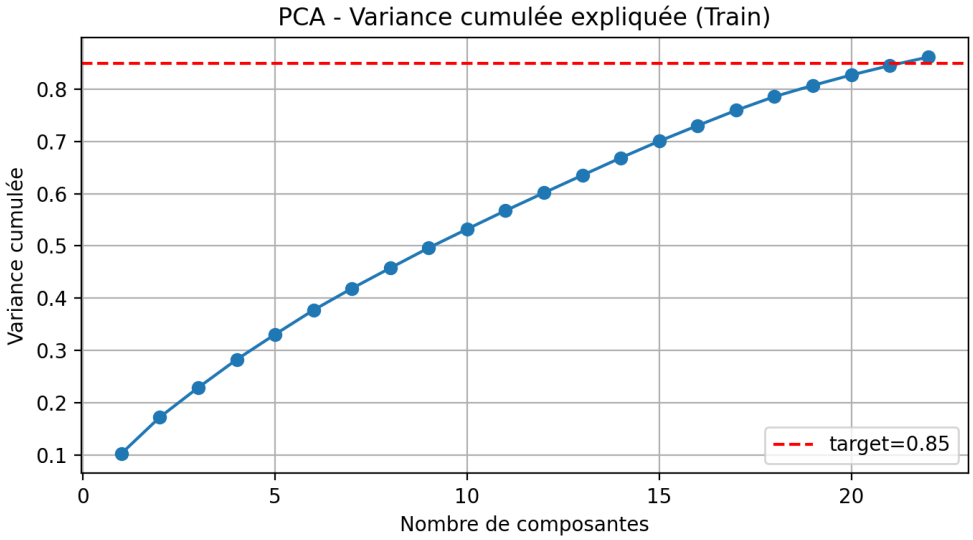

FIGURE 2 – Variance cumulée expliquée par les composantes principales.

Le seuil de variance cumulée retenu est de 85%. L'exécution finale montre qu'environ 22 composantes sont nécessaires pour atteindre ce seuil.

5.2 Projection 2D

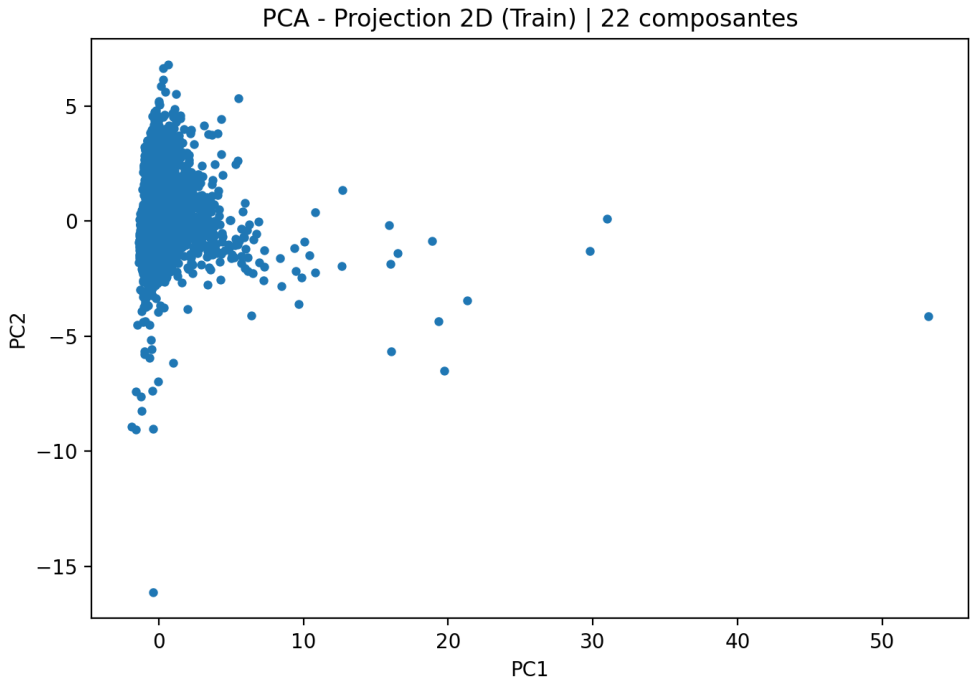

FIGURE 3 – Projection 2D des observations après PCA.

La projection PCA en deux dimensions montre une forte concentration centrale et quelques points plus éloignés. Cela suggère la présence d'un comportement moyen dominant et de quelques profils atypiques.

5.3 Segmentation KMeans

Après réduction de dimension, l'algorithme KMeans a été appliqué afin d'explorer une segmentation non supervisée des clients.

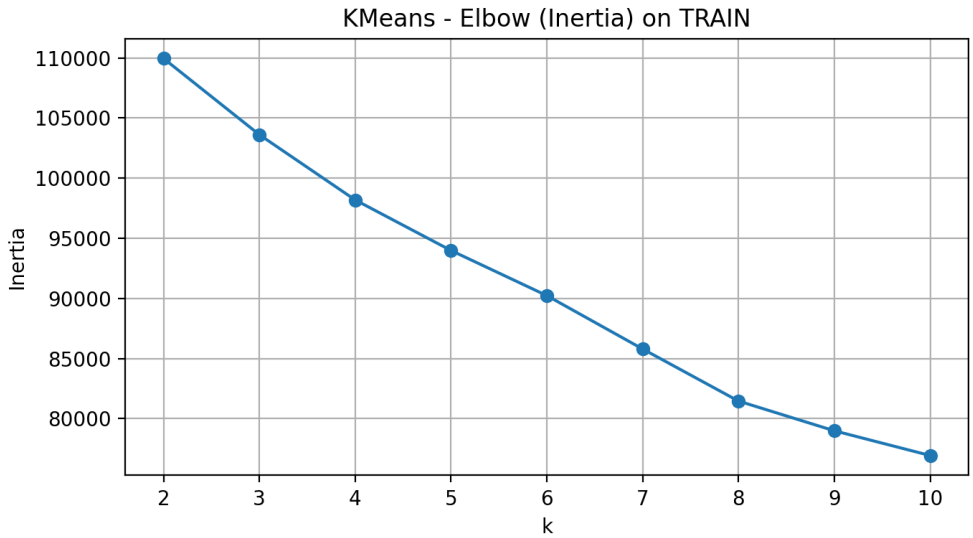

FIGURE 4 – Méthode du coude pour KMeans.

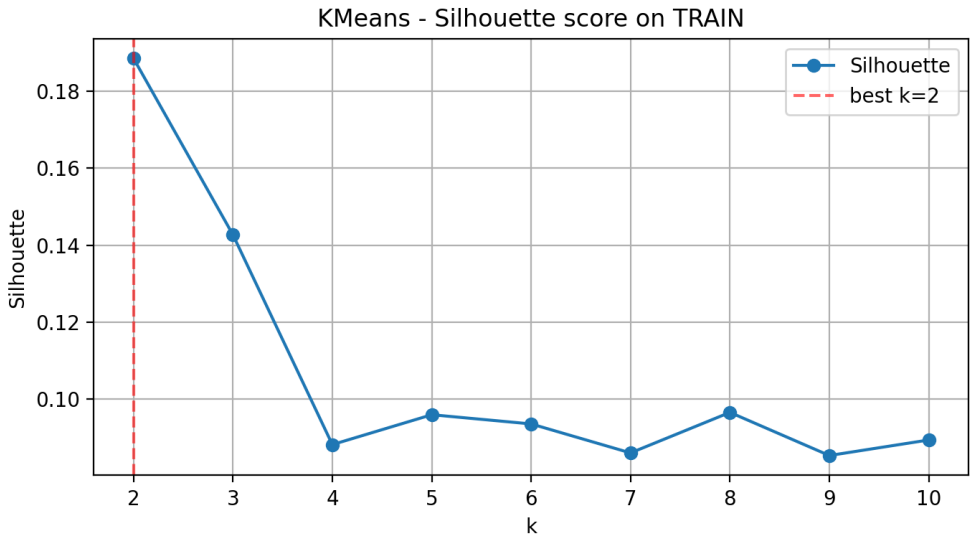

FIGURE 5 – Score de silhouette selon le nombre de clusters.

Le meilleur compromis est obtenu pour $k = 2$, ce qui signifie que la segmentation la plus stable met en évidence deux grands groupes plutôt qu'une partition fine.

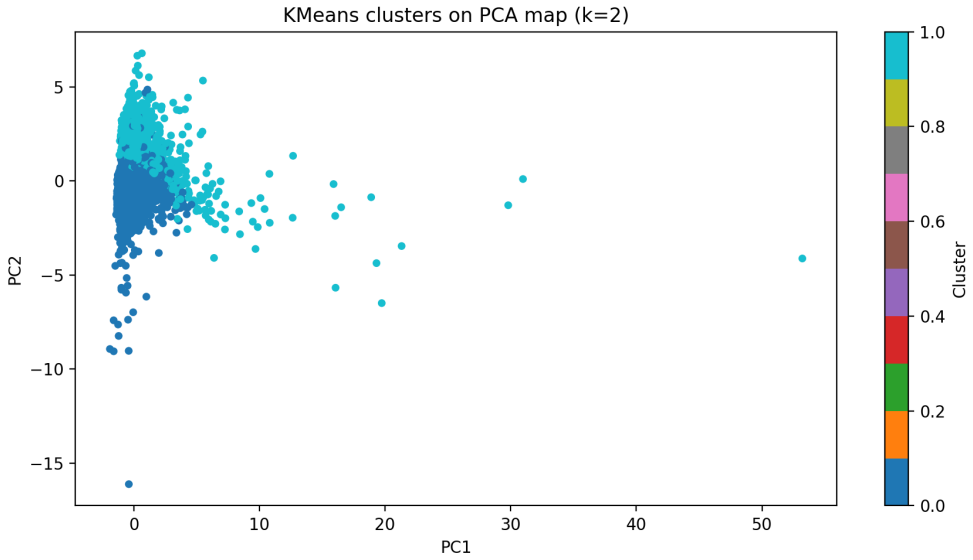

FIGURE 6 – Clusters KMeans projetés sur le plan PCA.

5.4 Interprétation

La segmentation KMeans reste principalement exploratoire. Elle permet de distinguer un groupe de clients au comportement plus central et un autre groupe plus dispersé. Néanmoins, la séparation visuelle reste modérée, ce qui confirme que la classification supervisée demeure l'axe principal du projet.

6 Modélisation prédictive : classification

6.1 Objectif

L'objectif principal de cette partie est de prédire le churn client à partir des variables préparées.

6.2 Modèles testés

Les modèles suivants ont été évalués :

- Logistic Regression
- Logistic Regression avec GridSearchCV
- Logistic Regression avec Optuna
- SVM
- SVM avec GridSearchCV
- Random Forest baseline
- Random Forest avec GridSearchCV

6.3 Métriques d'évaluation

Les métriques retenues sont :

- Accuracy
- F1-score
- ROC-AUC
- Matrice de confusion

6.4 Résultats de classification

Le meilleur modèle obtenu est `RandomForest_GridSearch`.

Table 1 – Synthèse des résultats de classification

Modèle	**Accuracy**	**F1-score**	**ROC-AUC**
LogisticRegression	0.8697	0.8202	0.9491
LogisticRegression_GridSearch	0.8697	0.8202	0.9491
LogisticRegression_Optuna	0.8686	0.8189	0.9491
SVM	0.8754	0.8222	0.9510
SVM_GridSearch	0.8914	0.8435	0.9585
RandomForest_baseline	0.8994	0.8483	0.9602
RandomForest_GridSearch	**0.9051**	**0.8566**	**0.9614**

6.5 Matrice de confusion

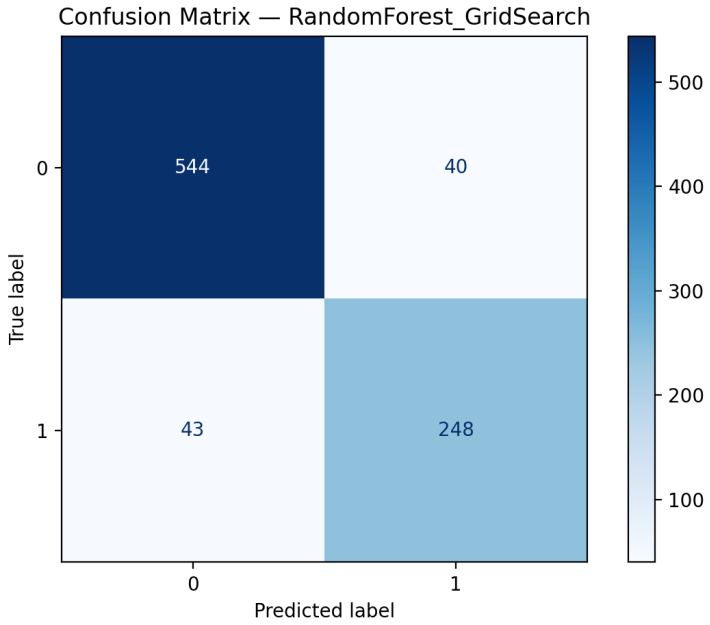

FIGURE 7 – Matrice de confusion du meilleur modèle de classification.

La matrice de confusion montre :

— 544 vrais négatifs ;

— 248 vrais positifs ;

— 40 faux positifs ;

— 43 faux négatifs.

Le modèle classe donc correctement la grande majorité des clients fidèles et des churners.

6.6 Interprétation

Les performances obtenues sont élevées mais réalistes. Le modèle final est plus robuste que les premières versions quasi parfaites, ce qui confirme que la phase de détection de fuite d'information et de sélection des variables a été déterminante.

6.7 Importance des variables

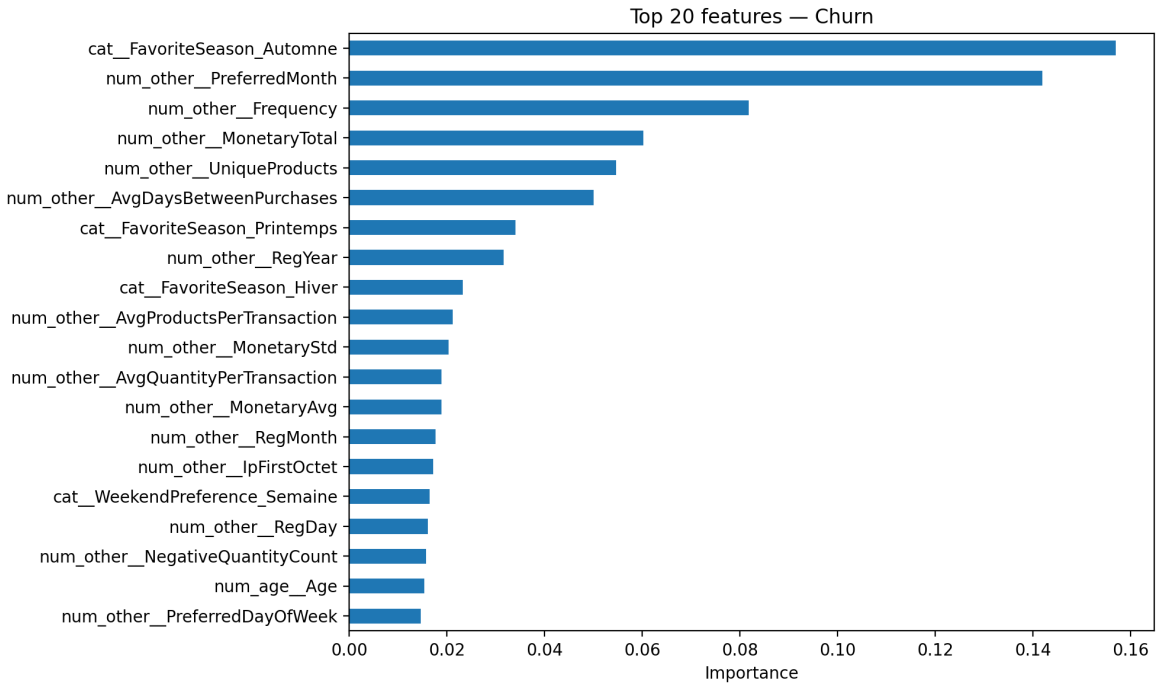

FIGURE 8 – Top 20 des variables les plus importantes pour la prédiction du churn.

L'analyse d'importance montre que le modèle repose principalement sur :

- la saisonnalité (`FavoriteSeason`, `PreferredMonth`) ;
- l'activité transactionnelle (`Frequency`, `MonetaryTotal`) ;
- la diversité produit (`UniqueProducts`) ;
- le rythme d'achat (`AvgDaysBetweenPurchases`) ;
- certaines composantes temporelles d'inscription.

7 Modélisation prédictive : régression

7.1 Objectif

L'objectif secondaire du projet est de prédire `MonetaryTotal`.

7.2 Modèles testés

Plusieurs modèles ont été comparés :

- Ridge (baseline linéaire)
- RandomForestRegressor
- ExtraTreesRegressor
- HistGradientBoostingRegressor

7.3 Métriques

Les métriques retenues sont :

— RMSE

— MAE

— R^2

7.4 Résultats

Le meilleur modèle obtenu est `HistGradientBoostingRegressor`.

Table 2 – Résultats de régression sur l'ensemble de test

Modèle	**RMSE**	**MAE**	R^2
Ridge	104233183.86	3524695.50	-102722685.4452
RandomForestRegressor	7535.86	568.82	0.4631
ExtraTreesRegressor	7913.41	610.46	0.4079
HistGradientBoostingRegressor	**6124.65**	**458.13**	**0.6453**

7.5 Analyse critique

Le modèle Ridge a présenté un R^2 fortement négatif, ce qui montre que l'hypothèse linéaire n'est pas adaptée à cette tâche. Les modèles non linéaires basés sur les arbres se sont révélés beaucoup plus performants. Le meilleur compromis est atteint avec `HistGradientBoostingRegressor`, qui capture mieux les relations complexes présentes dans les données.

8 Validation croisée et robustesse

8.1 Validation croisée en classification

Une validation croisée stratifiée en 5 plis a été réalisée pour les principaux modèles de classification.

Table 3 – Validation croisée en classification

Modèle	**Accuracy**	**F1-score**	**ROC-AUC**
LogisticRegression	0.8952 ± 0.0123	0.8985 ± 0.0118	0.9652 ± 0.0060
SVM	0.9062 ± 0.0119	0.9078 ± 0.0108	0.9716 ± 0.0071
RandomForest	**0.9207 ± 0.0058**	**0.9214 ± 0.0059**	**0.9797 ± 0.0040**

8.2 Validation croisée en régression

Une validation croisée en 5 plis a également été réalisée pour la régression.

Table 4 – Validation croisée en régression

Modèle	R^2	**RMSE**
RandomForestRegressor	0.5992 ± 0.0960	4078.34 ± 1820.85
ExtraTreesRegressor	0.6042 ± 0.1496	4107.01 ± 2035.26
HistGradientBoostingRegressor	**0.7667 ± 0.1201**	**3167.55 ± 1790.01**

8.3 Interprétation

Les résultats de validation croisée confirment la robustesse de la classification et l'amélioration nette de la régression. HistGradientBoostingRegressor apparaît comme le modèle le plus performant et le plus cohérent pour la tâche de régression.

9 Prédiction et inférence

9.1 Rôle du script de prédiction

Le script de prédiction recharge :

— le préprocesseur sauvegardé ;

— le meilleur modèle de classification ;

— les bornes IQR des outliers ;

— les informations de colonnes supprimées.

9.2 Pipeline d'inférence

Le pipeline d'inférence reproduit strictement les étapes apprises pendant l'entraînement :

— parsing des dates ;

— feature engineering sur l'adresse IP ;

— suppression des colonnes inutiles ou fuyantes ;

— application des bornes IQR ;

— alignement des colonnes avec celles attendues par le modèle ;

— prédiction finale et estimation de probabilité.

9.3 Résultats observés

Les tests sur profils réalistes montrent que le modèle distingue correctement les clients stables des clients à risque, même si certaines probabilités restent proches du seuil de décision, ce qui traduit une incertitude réaliste dans des cas intermédiaires.

10 Déploiement avec Flask

10.1 Objectif

Le déploiement vise à rendre le modèle accessible via une interface simple permettant à un utilisateur de tester différents profils clients.

10.2 Architecture

La solution repose sur :

— une interface HTML/CSS ;

— un backend Flask ;

— le préprocesseur sauvegardé ;

— le meilleur modèle entraîné ;

— les artefacts de prétraitement.

10.3 Modes d'utilisation

L'interface propose :

- un mode manuel à partir de 10 variables ;
- un mode test exact à partir d'une ligne réelle de `X_test.csv`.

10.4 Problèmes rencontrés

Plusieurs difficultés ont été rencontrées au cours du projet :

- fuite d'information dans certaines variables ;
- présence de colonnes trop redondantes ;
- déséquilibre de classes ;
- difficulté d'interprétation des modèles linéaires en régression ;
- écart entre le modèle complet et les entrées simplifiées de l'interface Flask ;
- erreurs de conversion des nombres saisis manuellement.

10.5 Corrections apportées

Ces problèmes ont été corrigés en :

- supprimant les variables suspectes ;
- rééquilibrant les classes avec SMOTENC ;
- choisissant des modèles non linéaires plus adaptés ;
- alignant strictement le pipeline d'inférence sur le pipeline d'entraînement ;
- testant l'interface sur des profils réels et manuels.

10.6 Limites

L'interface actuelle reste volontairement simplifiée. En mode manuel, seules 10 variables sont renseignées ; le reste doit être reconstruit ou imputé, ce qui peut introduire un léger décalage par rapport aux prédictions obtenues sur des lignes complètes.

11 Discussion générale

11.1 Points forts du projet

Les principaux points forts sont :

- une chaîne ML complète de bout en bout ;
- une gestion méthodologiquement correcte du data leakage ;
- un preprocessing rigoureux ;
- une comparaison de plusieurs modèles ;
- une optimisation par GridSearchCV et Optuna ;
- une validation croisée ;
- une phase d'inférence et une interface Flask.

11.2 Limites

Les limites principales du projet sont :

— le caractère essentiellement exploratoire de la segmentation KMeans ;

— une séparation probabiliste parfois modérée dans les cas limites ;

— un décalage possible entre le modèle complet et l'interface manuelle simplifiée.

11.3 Améliorations possibles

Plusieurs pistes d'amélioration peuvent être envisagées :

— enrichir l'interface avec davantage de variables ;

— calibrer les probabilités ;

— intégrer des outils d'explicabilité comme SHAP ;

— tester des modèles supplémentaires pour la régression ;

— développer un modèle dédié au mode manuel Flask.

12 Conclusion générale

Ce projet a permis de mettre en œuvre une démarche complète de Machine Learning appliquée à la prédiction du churn client dans un contexte retail. À partir d'un jeu de données riche mais imparfait, un pipeline robuste a été construit pour préparer les données, réduire la dimension, segmenter les clients, entraîner plusieurs modèles supervisés, comparer leurs performances et déployer une solution d'inférence.

Le meilleur modèle de classification obtenu est un `RandomForest_GridSearch`, qui atteint de très bonnes performances tout en restant cohérent après suppression des variables à risque de fuite. En parallèle, la partie régression a été significativement améliorée grâce à `HistGradientBoostingRegressor`, mieux adapté à la structure non linéaire des données. Enfin, une interface Flask a été développée pour démontrer la faisabilité d'un usage applicatif.

L'ensemble du travail montre qu'une préparation rigoureuse des données, une méthodologie soignée et une évaluation critique sont les fondements d'un projet ML crédible, professionnel et défendable.